HS Harz – Functional Safety

Airbag ECU

Hardware Concept Design Report

Crash Detection · Airbag Control · ISO 26262 Compliance

Document Type	Hardware Concept Design (HW Concept)
Project	Airbag ECU – LCP01
Version	1.00
Date	November 2025
Role	Hardware Concept Designer
Standard	ISO 26262:2018 – Road Vehicles Functional Safety

1. Purpose and Scope

This Hardware Concept Design Report documents the hardware architecture and design decisions for the Airbag Electronic Control Unit (ECU) developed as part of the HS Harz Functional Safety university project. The hardware concept covers the physical sensor interfaces, microcontroller selection, power supply design, output driver stages, CAN communication hardware, and fault-detection circuitry, all aligned with the requirements of ISO 26262.

The prototype platform is the Arduino Due (ARM Cortex-M3, 84 MHz), chosen for its 32-bit processing capability and available CAN library support (due_can). This report defines the hardware blocks that support the software requirements specified in the FuSa Software Report and maps each hardware element to its corresponding safety function.

2. System Overview

The Airbag ECU (ACU) is a safety-critical embedded system responsible for detecting crash events and commanding airbag deployment within the automotive environment. The hardware must reliably process sensor signals, make deployment decisions, and communicate fault states — all within the tight timing constraints of a real-world crash event (typically < 30 ms from impact to deployment).

2.1 Operating Environment

Parameter	Specification
Temperature Range	−40 °C to +85 °C (automotive grade)
Mechanical Shock	Per LV124 standard
EMI Requirements	ECE R10 compliant
Supply Voltage	9–16 V DC (nominal 12 V)
Post-Crash Power	Capacitor-backed (≥ 150 ms hold-up)
Communication Bus	CAN 2.0B, ID 0x120, 8 bytes

2.2 Safety-Relevant Functions

The hardware must support the following safety-relevant system functions defined in the system concept:

- F1 – Crash detection via acceleration, gyroscope, and pressure sensors
- F2 – Airbag squib firing (simulated via digital output in prototype)
- F3 – Sensor monitoring and plausibility checking
- F4 – System self-test and diagnostics
- F5 – Fail-safe / fall-back on detected hardware faults
- F6 – CAN communication of safety-relevant states to vehicle ECU

3. Hardware Architecture

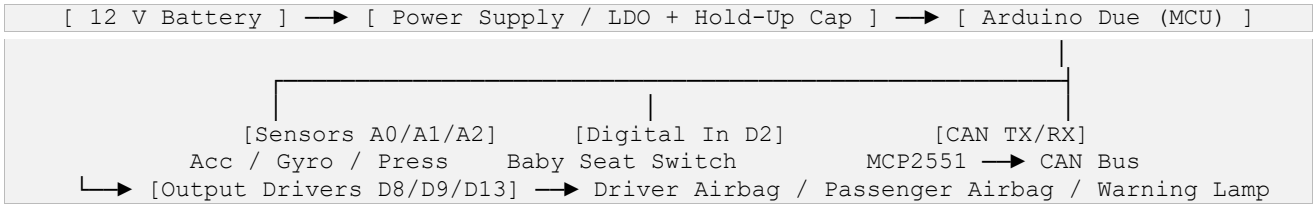
3.1 Architecture Block Description

The hardware is organised into six primary functional blocks that form the hardware architecture of the Airbag ECU:

Block	Component (Prototype)	Function
Power Supply	12 V → 5 V / 3.3 V LDO	Provides regulated voltage; capacitor hold-up for post-crash operation
Microcontroller	Arduino Due (ARM Cortex-M3)	Central processing: sensor reading, crash logic, output control, CAN
Sensor Inputs	Analog potentiometers (simulation)	Frontal acceleration (A0), gyroscope/rollover (A1), side pressure (A2)
Digital Input	Push button / switch (D2)	Baby/passenger seat detection signal (INPUT_PULLUP)
Output Drivers	LEDs on D8, D9, D13	Driver airbag, passenger airbag (simulated), warning lamp
CAN Interface	CAN0 (Due built-in) + MCP2551 transceiver	ISO 11898 CAN bus communication; diagnostic frame transmission and reception

3.2 Hardware Architecture Diagram (Block Level)

The following diagram describes the signal flow and hardware block interconnections of the Airbag ECU:



4. Microcontroller Selection

4.1 Selected Device: Arduino Due (Atmel SAM3X8E)

The Arduino Due is selected as the prototype microcontroller platform. It is based on the Atmel SAM3X8E ARM Cortex-M3 processor and provides adequate processing power and peripheral count for the airbag ECU prototype demonstration.

Feature	Specification
CPU Core	ARM Cortex-M3, 32-bit
Clock Speed	84 MHz
Flash Memory	512 KB
SRAM	96 KB
Digital I/O Pins	54 (of which 12 PWM capable)
Analog Input Pins	12 (10-bit ADC, 0–3.3 V)
CAN Controllers	2× on-chip CAN 2.0B (CAN0, CAN1)
Operating Voltage	3.3 V I/O logic (5 V tolerant inputs)
USB Interface	Native USB + Programming USB

4.2 Rationale for Selection

- Dual on-chip CAN controllers allow full CAN 2.0B communication without additional CAN controller ICs
- 32-bit ARM Cortex-M3 provides sufficient computational margin for real-time crash detection loops
- 12-channel ADC supports simultaneous reading of multiple analog sensors
- 54 GPIO pins provide adequate I/O for sensor inputs, output drivers, and diagnostic signals
- due_can library provides ready-to-use interrupt-driven CAN support, accelerating prototype development
- Wide community support and simulation availability (Wokwi, Tinkercad) enable rapid prototyping and verification

4.3 Production Hardware Note

In a production-grade Airbag ECU, a qualified automotive-grade microcontroller (e.g., Infineon AURIX TC3xx, NXP S32K, or Renesas RH850) with hardware safety mechanisms including lockstep CPU cores, ECC memory, hardware watchdog, and SIL/ASIL-certified software toolchains would be selected. The Arduino Due is used exclusively as a functional prototype.

5. Sensor Interface Hardware

5.1 Analog Sensor Inputs

Three analog sensor channels provide the crash detection inputs. In the prototype, potentiometers (variable resistors) are used to simulate sensor outputs. In a production design, MEMS-based

sensors with SPI or I2C digital interfaces would be used, or analog sensors with signal conditioning amplifiers.

Sensor	Arduino Pin	Signal Type	ADC Range	Crash Threshold
Acceleration Sensor (frontal crash)	A0	0–1023 (10-bit ADC)	≥ 700 (frontal crash)	Triggers frontal crash condition
Gyroscope Sensor (rollover / safing)	A1	0–1023 (10-bit ADC)	≥ 600 (safing condition)	Required as safing channel (dual-point logic)
Pressure Sensor (side impact)	A2	0–1023 (10-bit ADC)	≥ 650 (side crash)	Triggers side crash condition

5.2 Sensor Signal Conditioning (Production Design Recommendations)

In a production Airbag ECU, the following hardware conditioning stages would be required between the sensor and the ADC input:

- Anti-aliasing low-pass filter (RC filter, fc ≈ 500 Hz) to prevent ADC aliasing from high-frequency vibration noise
- ESD protection diodes on all analog input lines
- Operational amplifier buffer stage to provide impedance matching between sensor output and ADC input
- Voltage divider or level-shift circuit where sensor output voltage range exceeds ADC reference (3.3 V on Due)
- Common-mode choke or ferrite bead for EMI suppression per ECE R10

5.3 Digital Input: Baby/Passenger Seat Switch

Parameter	Detail
Arduino Pin	D2 (Digital Input)
Logic Type	Active LOW with internal INPUT_PULLUP enabled
LOW (0 V)	Baby/child seat detected → passenger airbag suppressed
HIGH (3.3 V)	No baby seat detected → passenger airbag deployment permitted
Fault Condition	Signal outside valid range or intermediate voltage → safe state

5.4 Sensor Validity Monitoring

The hardware design must support plausibility checking of sensor outputs as required by safety function F3. The following hardware-level validity checks are defined:

- ADC value > 1020 or < 2 → sensor short-circuit or open-circuit fault detected → safe state entry
- Multiple sensors simultaneously at maximum reading → implausible condition → flag for software plausibility check
- Gyroscope reading absent while acceleration indicates crash → safing channel fault → block deployment

6. Output Driver Hardware

6.1 Output Signal Overview

Output Function	Pin	Logic Active	Description
Driver Airbag Deployment	D8	HIGH (3.3 V)	Activates when valid crash + safing condition TRUE
Passenger Airbag Deployment	D9	HIGH (3.3 V)	Activates only when crash valid AND baby seat NOT detected
Warning Lamp (Fault Indicator)	D13	HIGH (3.3 V)	Illuminated during safe state or detected fault condition

6.2 Prototype Output Implementation

In the prototype, LED indicators connected to D8, D9, and D13 simulate the airbag squib firing outputs and the fault warning lamp. The LEDs are driven directly from the Arduino Due GPIO pins with a current-limiting resistor (typically 220–470 Ω for 3.3 V logic).

6.3 Production Output Driver Design

In a production Airbag ECU, the airbag squib firing circuit requires dedicated hardware safety mechanisms to prevent unintended deployment:

- High-side and low-side MOSFET switches in series (safing switch architecture) — both must be independently commanded HIGH before current can flow through the squib
- Firing capacitor (typically 47–100 µF, 50 V) charged from the main supply, providing energy for deployment even in post-crash battery disconnection scenarios
- Squib resistance monitoring circuit to detect open-circuit (broken wire) or short-circuit (shorted squib) faults
- Hardware current limiter to prevent over-current destruction of the firing transistors
- Independent safing channel from a separate hardware comparator (not via the MCU) — this prevents deployment if only the microcontroller malfunctions
- Fire Enable signal from the MCU must be accompanied by the hardware safing enable for deployment to occur (two-point safety architecture)

6.4 Output Readback Verification

The software performs output readback after commanding each output (SWR-10). The hardware must support this by routing the output pin state back to a readable input. In production, this is achieved through:

- Dedicated feedback input line from the output driver stage
- Voltage comparator monitoring the squib driver output voltage
- If commanded HIGH but readback reads LOW → output fault → immediate safe state entry

7. Power Supply Design

7.1 Supply Architecture

The Airbag ECU power supply must remain functional during and after a crash event. The vehicle battery voltage may drop significantly during a crash due to battery disconnection or cable damage. The supply hardware must provide hold-up energy to complete airbag deployment.

Supply Stage	Specification
Input Voltage Range	9–16 V DC (vehicle battery nominal 12 V)
Primary Regulator	5 V LDO or switching DC-DC converter for MCU supply
MCU Core Supply	3.3 V (Arduino Due operates at 3.3 V logic)
Post-Crash Hold-Up	Capacitor bank (≥ 150 ms at full load) — powers squib firing and MCU after battery loss
Reverse Polarity Protection	P-channel MOSFET or Schottky diode at battery input
Overvoltage Protection	TVS diode at input to clamp transients (load dump: up to 40 V per ISO 7637-2)
Prototype Supply	Arduino Due powered via USB (5 V) with onboard 3.3 V LDO; no crash hold-up in prototype

7.2 Prototype Power Note

In the prototype simulation environment (Wokwi/Tinkercad), the Arduino Due is powered via USB. The crash hold-up capacitor and reverse polarity protection are not implemented in the prototype but are described here as production hardware requirements.

8. CAN Communication Hardware

8.1 CAN Bus Interface

The Airbag ECU uses CAN (Controller Area Network) to transmit crash status, fault codes, and diagnostic information to the main vehicle ECU. CAN was selected for its deterministic timing, differential signalling (noise immunity), and widespread adoption in automotive ECUs.

CAN Parameter	Value
CAN Standard	ISO 11898 / CAN 2.0B
CAN Controller	On-chip CAN0 (Arduino Due SAM3X8E)
CAN Transceiver (Production)	MCP2551 or TJA1050 (5 V, ISO 11898 compliant)
Message ID	0x120 (high priority airbag frame)
Data Length Code (DLC)	8 bytes
Typical Bus Speed	500 kbit/s (automotive standard)
Termination	120 Ω at each end of the CAN bus

8.2 CAN Frame Structure

Byte	Field Name	Content / Meaning

Byte 0	System Status	0x00 = Normal, 0x01 = Crash detected, 0xFF = Fault
Byte 1	Acceleration Value	Raw ADC reading from A0 (0–255 scaled)
Byte 2	Gyroscope Value	Raw ADC reading from A1 (0–255 scaled)
Byte 3	Pressure Value	Raw ADC reading from A2 (0–255 scaled)
Byte 4	Seat Status	0x00 = No baby seat, 0x01 = Baby seat detected
Byte 5	Deployment Status	Bit 0 = Driver airbag, Bit 1 = Passenger airbag
Byte 6	Fault Code	0x00 = No fault, 0x01–0xFF = Specific fault codes
Byte 7	Checksum	XOR checksum of Bytes 0–6 for frame integrity

8.3 CAN Hardware Safety

- CAN bus error frame detection — hardware CAN controller flags bus-off state → MCU enters safe state
- CAN transceiver TXD dominant timeout protection (MCP2551 feature) prevents bus locking if MCU hangs
- CAN wake-up from sleep mode via bus activity supports low-power standby (not required in prototype)

9. Hardware Safe State Design

9.1 Safe State Definition

The hardware safe state ensures that airbag deployment cannot occur accidentally when a hardware fault is detected. The safe state is defined as:

- Driver airbag output (D8) = LOW (deployment disabled)
- Passenger airbag output (D9) = LOW (deployment disabled)
- Warning lamp output (D13) = HIGH (fault indicated)
- CAN fault frame transmitted with fault code in Byte 6

9.2 Safe State Trigger Conditions

ID	Trigger Condition	Hardware Implication
FS-01	ADC value out of valid range (< 2 or > 1020)	Sensor open/short circuit; hardware fault — block deployment
FS-02	Output readback mismatch (commanded HIGH, reads LOW)	Output driver failure; hardware fault — activate warning lamp
FS-03	CAN bus error detected (bus-off state)	Communication hardware fault — enter safe state
FS-04	Baby seat signal intermediate voltage (neither 0 V nor 3.3 V)	Seat switch hardware fault — passenger airbag suppressed, warn
FS-05	Watchdog timer expiry (MCU stall)	Hardware watchdog resets MCU; outputs go to default LOW (safe)

9.3 Hardware Watchdog

A hardware watchdog timer must be implemented to detect MCU software stalls or infinite loops. If the software fails to pet (reset) the watchdog within the defined timeout window (typically 10–100 ms), the watchdog hardware resets the MCU. On reset, all output pins default to LOW — this is the inherent safe state of the digital outputs.

10. Hardware Requirements Traceability

The following table maps each Software Requirement (SWR) to its corresponding Hardware Design element, demonstrating full traceability from requirement to hardware implementation.

SWR ID	Software Requirement	Supporting Hardware Design Element
SWR-01	Read acceleration sensor input	Analog input A0, 10-bit ADC, potentiometer simulation (production: MEMS accelerometer)
SWR-02	Read gyroscope angular sensor input	Analog input A1, 10-bit ADC (production: MEMS gyroscope with signal conditioning)
SWR-03	Read pressure sensor input	Analog input A2, 10-bit ADC (production: MEMS pressure sensor + low-pass filter)
SWR-04	Read passenger seat digital input	Digital input D2 with INPUT_PULLUP; active-LOW logic

SWR-05	Activate driver airbag output	Digital output D8 (production: high/low-side MOSFET squib driver)
SWR-06	Activate passenger airbag (no baby seat)	Digital output D9 with D2 seat switch interlock logic
SWR-07	Activate warning lamp during fault	Digital output D13 (active HIGH)
SWR-08	Send crash/fault status via CAN	Arduino Due CAN0 controller + MCP2551 transceiver, ID 0x120
SWR-09	Read CAN messages for diagnostic/reset	CAN0 receive interrupt handler; due_can library
SWR-10	Read back output status after activation	GPIO readback of D8/D9 after asserting; hardware feedback path in production
SWR-11	Enter safe state if fault detected	Hardware watchdog, output default-LOW on reset, warning lamp D13
SWR-12	Support test routines for module/integration testing	Serial/USB debug interface; Wokwi simulation platform for hardware-in-loop testing

11. Hardware Test and Verification Plan

11.1 Hardware Test Cases

Test ID	Test Type	Test Condition	Expected Result	Maps to SWR
HT-01	Sensor Input	A0=300 (below threshold)	No crash flag, airbags OFF	SWR-01
HT-02	Sensor Input	A0=800, A1=650 (frontal + safing)	Crash flag set, D8 HIGH	SWR-01, 05
HT-03	Sensor Input	A2=800, A1=650 (side + safing)	Crash flag set, D8 HIGH, D9 HIGH	SWR-03, 05, 06
HT-04	Digital Input	D2 LOW (baby seat), A0=800, A1=650	D8 HIGH, D9 LOW (suppressed)	SWR-04, 06
HT-05	Safe State	A0 > 1020 (sensor out of range)	D13 HIGH, D8/D9 LOW, CAN fault TX	SWR-07, 11
HT-06	Output Readback	Command D8 HIGH, read D8	Readback matches command	SWR-10
HT-07	CAN TX	Valid crash inputs applied	8-byte frame on CAN, ID 0x120	SWR-08
HT-08	Safing Only	A1=700 (safing only, no crash sensor)	No deployment (safing alone insufficient)	SWR-01, 05

11.2 Hardware Coverage Goals

Coverage Type	How Achieved in Hardware Testing
Input Range Coverage	Sweep analog inputs across full ADC range (0–1023) to verify threshold detection
Output State Coverage	Verify each digital output (D8, D9, D13) activates and deactivates correctly per logic conditions
Fault Injection Coverage	Apply out-of-range sensor values and verify safe state entry (warning lamp ON, airbags OFF)
CAN Frame Integrity	Capture and decode CAN frames; verify all 8 bytes match expected values for each test scenario
Safe State Entry	Confirm that every fault trigger condition results in safe state activation within one control loop cycle

12. Glossary

Term / Abbreviation	Definition
ACU	Airbag Control Unit — the ECU responsible for crash detection and airbag deployment control
ADC	Analog-to-Digital Converter — converts analog sensor voltage to a digital integer value

ARM Cortex-M3	32-bit RISC processor core used in the Arduino Due (SAM3X8E)
ASIL	Automotive Safety Integrity Level — A to D, D being highest requirement (ISO 26262)
CAN	Controller Area Network — automotive serial communication bus standard (ISO 11898)
ECU	Electronic Control Unit — embedded computing module in a vehicle
ESD	Electrostatic Discharge — overvoltage event that can damage electronic components
EMI	Electromagnetic Interference — unwanted electrical noise affecting circuit operation
GPIO	General Purpose Input/Output — configurable digital pin on a microcontroller
HW	Hardware
ISO 26262	International standard for functional safety of road vehicle electrical/electronic systems
LDO	Low Drop-Out Regulator — linear voltage regulator with low input-output voltage differential
MCU	Microcontroller Unit
MEMS	Micro-Electro-Mechanical System — miniature sensor technology used for accelerometers and gyroscopes
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor — used as switching element in output drivers
Safing Condition	A secondary, independent confirmation of crash event required before deployment is enabled (dual-point safety architecture)
SWR	Software Requirement — functional requirement assigned to the software module
SW	Software
TVS	Transient Voltage Suppressor — protection diode for input overvoltage clamping

13. List of Dependent Documents

No.	Document Name	Version	Date	File Name
1	FuSa Software Report – Arduino Coding and Logic Implementation for Airbag ECU System	1.00	Nov 2025	FuSa_Report_Arduino.pdf
2	Airbag ECU Portfolio Template (HS Harz)	1.00	23.11.2025	20251123_LCP01_AirbagEcu_template.doc
3	ISO 26262:2018 – Road Vehicles – Functional Safety	2nd Ed.	2018	ISO Standard

4	LV124 – Electrical and Electronic Components in Motor Vehicles	—	—	LV124
5	ECE R10 – Electromagnetic Compatibility of Vehicles	—	—	ECE R10
6	Wokwi Simulation Project – Airbag ECU Prototype	—	2025	wokwi.com/projects/463574030977197057

14. Version History

Version	Date	Author	Changes
1.00	November 2025	Hardware Concept Designer	Initial release — full hardware concept design based on FuSa Software Report and HS Harz template